

Project Documentation

Version 4.0 - Vectorization Phase

1. Project Overview

This project implements a Canny Edge Detection pipeline in C++, specifically optimized for and cross-compiled to the RISC-V (RV64GCV) architecture. The project follows a strict Test-Driven Development (TDD) methodology, ensuring mathematical correctness before deploying to the target hardware via QEMU emulation. The ultimate goal is to benchmark and optimize the image processing algorithms using hardware-specific features, such as the RISC-V Vector Extension (RVV).

2. Architecture & Pipeline Components

The software has been expanded from a filtering baseline to a complete computer vision pipeline. All stages are implemented and currently being optimized:

- **Gaussian Blur (gaussian.h):** Applies a 5x5 convolution kernel to smooth the image.
- **Sobel Filter (sobel.h):** Computes the spatial gradient of the image using 3x3 kernels.
- **Gradient Magnitude (magnitude.h):** Combines gradients using L1/L2 norms and normalizes output to [0, 255].
- **Gradient Direction (direction.h):** Quantizes edges into four primary bins (0°, 45°, 90°, 135°).
- **Non-Max Suppression (nms.h):** Thins edges by comparing magnitudes along the gradient direction.
- **Double Thresholding (threshold.h):** Classifies pixels as STRONG, WEAK, or NON-EDGE; includes auto_threshold logic.
- **Hysteresis (hysteresis.h):** Uses a stack-based flood fill to finalize edges.

3. Development Phases & Milestones

Phase 1-3: Foundation and Verification

Environment setup and scalar implementations are complete. All modules, including the new NMS and Hysteresis stages, have accompanying unit tests in `test_pipeline.cpp` to ensure algorithmic integrity across all pipeline additions.

Phase 4: Compiler Optimization Sweep

Updated Status: Completed. The full 7-stage pipeline has been successfully benchmarked. The introduction of `nms.h`, `threshold.h`, and `hysteresis.h` added branch-heavy logic and standard library dependencies (`std::stack`) that were thoroughly profiled across all optimization levels (`-O0`, `-O2`, `-O3`, `-Os`, `-Ofast`).

Phase 5: Profiling & Bottleneck Analysis

Updated Status: Completed. The timing harness in `main.cpp` was expanded to measure the millisecond execution time of the entire pipeline, specifically identifying how much cycle time is consumed by the new non-convolutional stages. Total scalar execution time for a 256x256 image at `-O3` is approximately 2282.16 ms.

4. Build System & Usage

The project utilizes a Makefile to manage compilation for both the native host and the RISC-V target.

Host Machine Testing

Validates mathematical logic using Google Test:

```
make clean make test
```

RISC-V Emulation

Cross-compiles and runs on the QEMU emulator:

```
make clean make run
```

Optimization Sweeps

Benchmarks full pipeline performance:

```
make run OPT=-O3
```

5. Optimization Results (Phase 6 - RVV Vectorization)

The highest-latency modules identified in profiling are now being accelerated using explicit SIMD vector intrinsic functions (<rvv_vector.h>).

Optimized Stage	Scalar Time (-O3)	RVV Time (VLEN=128)	Speedup (x)
Gaussian Blur (5x5)	~3800 ms	1127.91 ms	3.37x
Magnitude	~650 ms	199.06 ms	3.26x

6. Current Status & Next Steps

Status: The full scalar pipeline is mathematically verified. Phases 4 and 5 are complete, providing a complete performance baseline for all 7 modules. Phase 6 (Vectorization) is actively underway, with Gaussian Blur and Magnitude successfully optimized.

Next Steps (Handover):

The project is transitioning to the next set of optimizations. The immediate targets for explicit SIMD vectorization are:

- Sobel Filter:** Vectorize the spatial gradient computations.
- Non-Max Suppression:** Optimize the branch-heavy logic using RVV vector masks.